~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/flightCompV2.ino

```
/*
 * Program to combine all ARC-S Files into a flight profile for the ARC-S 2026 Fair
 *
 * Name:      Thor Kopenkoskey
 * Created:   November 17, 2025
 * Edited:    Jan 25, 2025
 */

#include "telemetry.h"
#include <sensorfusion.h>
#include <servos.h>
#include "flash.h"
#include "sdcard.h"
Telemetry telemetry;

#define SerialPort Serial
#define I2C_SDA 9
#define I2C_SCL 46

extern craftState vehicle;

void setup()
{
  delay(300);
  Serial.begin(115200);
  delay(2000);
  Serial.println("Serial OK");

  EEPROM.begin(EEPROM_SIZE);

  Wire.begin(I2C_SDA, I2C_SCL);
  Wire.setClock(100000);
  Wire.setTimeout(1000);
  delay(100);
  Wire.beginTransmission(0x00);  // General call address (should always NACK, but tests bus)
  uint8_t error = Wire.endTransmission();
  if (error != 0 && error != 2) {  // 2 = NACK (expected), other errors indicate problems
    Serial.print("WARNING: I2C bus test returned error code: ");
    Serial.println(error);
  }

  SPI.begin();

  vehicle.initalizeSensors();

```

```
  for (int i = 0; i < NUM_SERVOS; i++) {
    servos[i].init_servo();
  }

  loadOffsetsFromEEPROM();  // restore trims on startup
  Serial.println("Servo system ready.");
}

void loop()
{
  double dt = getDeltaTime();
  unsigned long currentTime = micros();

  readSerialAndUpdateServo();

  vehicle.readGAM();
  vehicle.readBarometerData();
  pitchYawControlLoop();

  if (vehicle.gyro_dps.z > 90)
  {
    rollControlLoop();
  }

  if (vehicle.launch == false)
  {
    vehicle.preFlightPositionalTracking(dt);
    vehicle.preFlightOrientationTracking(dt);

    if (vehicle.gyro_dps.z > 1.5)
    {
      vehicle.launch = true;
    }
  }

  if (vehicle.launch == true)
  {

    vehicle.duringFlightPositionalTracking(dt);
    vehicle.duringFlightOrientationTracking(dt);
  }

    telemetry.serialIMU();
    telemetry.serialGyro();
    telemetry.serialBarometer();
    Serial.println();  // Blank line between samples
}
```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/sensorfusion.cpp

```cpp
/*
 * Program to combine all ARC-S Sensors into positional tracking/calculations.
 *
 * Name:      Thor Kopenkoskey
 * Created:   July 1, 2025
 * Edited:    Jan 27, 2025
 */

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BusIO_Register.h>
#include <Adafruit_LIS3MDL.h>
#include <Arduino_LSM6DSOX.h>
#include <cmath>
#include <BMP388_DEV.h>
#include <sensorfusion.h>

  const double GRAVITY = 9.80665;
  craftState vehicle;
  Adafruit_LIS3MDL magnetometer;
  BMP388_DEV bmp388;

void craftState::initalizeSensors()
{
  bmp388.begin();                                 // Default initialisation, place the BMP388 into SLEEP_MODE
  bmp388.setTimeStandby(TIME_STANDBY_1280MS);     // Set the standby time to 1.3 seconds
  bmp388.startNormalConversion();

  if (!IMU.begin())
  {
    Serial.println("IMU failed!");
  }
  else
  {
    Serial.println("IMU OK");
  }

  if (!magnetometer.begin_I2C(LIS3MDL_ADDR, &Wire))
  {
    Serial.println("Magnetometer failed!");
  }
  else
  {
    Serial.println("Mag OK");
  }
```

```
}

double getDeltaTime()
{
  static unsigned long lastTime = 0;         // remembers last call
  unsigned long currentTime = micros();      // current time in microseconds

  double dt = (currentTime - lastTime) / 1e6; // convert to seconds, sensors report MS and DPS

  lastTime = currentTime;

  return dt;
}

void craftState::readBarometerData()
{
  float t, p, a;

  if (bmp388.getMeasurements(t, p, a))
  {
    vehicle.temperature_C = t;
    vehicle.pressure_Pa   = p;
    vehicle.altitude_m    = a;
  }

}

//GAM = Gyro, Accel, Magno
void craftState::readGAM()
{
  float ax, ay, az;
  float gx, gy, gz;

  if (IMU.accelerationAvailable())
  {
    IMU.readAcceleration(ax, ay, az);
  }
  if (IMU.gyroscopeAvailable())
  {
    IMU.readGyroscope(gx, gy, gz);
  }

  vehicle.accel_g.x = ax;
  vehicle.accel_g.y = ay;
  vehicle.accel_g.z = az;

  vehicle.gyro_dps.x = gx;
  vehicle.gyro_dps.y = gy;
```

```
  vehicle.gyro_dps.z = gz;

  sensors_event_t magEvent;
  magnetometer.getEvent(&magEvent);

  vehicle.mag_uT.x = magEvent.magnetic.x;
  vehicle.mag_uT.y = magEvent.magnetic.y;
  vehicle.mag_uT.z = magEvent.magnetic.z;
}

void craftState::duringFlightOrientationTracking(double dt)
{
    // Integrate to get angles, LSOX Sensor gives rotation velocity
    vehicle.gyroAngle_deg.x += vehicle.gyro_dps.x * dt;
    vehicle.gyroAngle_deg.y += vehicle.gyro_dps.y * dt;
    vehicle.gyroAngle_deg.z += vehicle.gyro_dps.z * dt;
}

void craftState::duringFlightPositionalTracking(double dt)
{
  vehicle.deadReckoning_mps2.x += (vehicle.accel_g.x * GRAVITY * dt);
  vehicle.deadReckoning_mps2.y += (vehicle.accel_g.y * GRAVITY * dt);
  vehicle.deadReckoning_mps2.z += (vehicle.accel_g.z * GRAVITY * dt);
}

void craftState::preFlightOrientationTracking(double dt)
{
  const float alpha = 0.50;
  //Corect pitch/roll with IMU while static
  vehicle.gyroAngle_deg.x = alpha * vehicle.gyroAngle_deg.x + (1 - alpha) *
(vehicle.pitch * 180.0 / PI);
  vehicle.gyroAngle_deg.y = alpha * vehicle.gyroAngle_deg.y + (1 - alpha) *
(vehicle.roll * 180.0 / PI);

  //Correct yaw with fusion Magnometer
  vehicle.yaw = atan2(vehicle.mag_uT.y, vehicle.mag_uT.x) * (180.0 / PI); //
degrees
  if (vehicle.yaw < 0) vehicle.yaw += 360.0; // Normalize to 0-360

  vehicle.gyroAngle_deg.z = alpha * vehicle.gyroAngle_deg.z + (1 - alpha) *
vehicle.yaw;;
}

void craftState::preFlightPositionalTracking(double dt)
{
  vehicle.deadReckoning_mps2.x += vehicle.gyroAngle_deg.x / dt;
  vehicle.deadReckoning_mps2.y += vehicle.gyroAngle_deg.y / dt;
  vehicle.deadReckoning_mps2.z += vehicle.gyroAngle_deg.z / dt;

  vehicle.computeGravity(); //Gives Gravity IMU Offset
```

```
140
141   vehicle.deadReckoning_mps2.x = (vehicle.accel_g.x * 9.81) -
vehicle.gravity_mps2.x;
142   vehicle.deadReckoning_mps2.y = (vehicle.accel_g.y * 9.81) -
vehicle.gravity_mps2.y;
143   vehicle.deadReckoning_mps2.z = (vehicle.accel_g.z * 9.81) -
vehicle.gravity_mps2.z;
144 }
145 void craftState::computeGravity()
146     {
147         // Compute pitch and roll from accelerometer
148         pitch = atan2(accel_g.x, sqrt(accel_g.y*accel_g.y + accel_g.z*accel_g.z));
149         roll  = atan2(accel_g.y, sqrt(accel_g.x*accel_g.x + accel_g.z*accel_g.z));
150
151         // Gravity vector in sensor frame (m/s²)
152         gravity_mps2.x = 9.81 * sin(pitch);
153         gravity_mps2.y = 9.81 * sin(roll);
154         gravity_mps2.z = sqrt(9.81*9.81 - gravity_mps2.x*gravity_mps2.x -
gravity_mps2.y*gravity_mps2.y);
155     }
```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/sensorfusion.h

```
/*
 * Program to combine all ARC-S Sensors into positional tracking/calculations.
 *
 * Name:      Thor Kopenkoskey
 * Created:   November 17, 2025
 * Edited:    Jan 28, 2025
 */

#pragma once
#ifndef SENSORFUSION_H
#define SENSORFUSION_H
#include <Arduino_LSM6DSOX.h>
#include <Adafruit_LIS3MDL.h>
#include <BMP388_DEV.h>

#define LSM6DSOX_ADDR 0x6B
#define LIS3MDL_ADDR 0x1E

struct Vec3
{
    double x = 0.0;
    double y = 0.0;
    double z = 0.0;
    void zero();
};

class craftState
{
public:
    // Raw sensors
    Vec3 accel_g;
    Vec3 gyro_dps;
    Vec3 mag_uT;

    // Derived state
    Vec3 deadReckoning_mps2;
    Vec3 gyroAngle_deg;
    Vec3 gravity_mps2;

    double pitch = 0.0;
    double roll = 0.0;
    double yaw = 0.0;

    float temperature_C;
    float pressure_Pa;
    float altitude_m;

```

```
    bool launch = false;
    bool apogee = false;
    bool landed = false;

    void initalizeSensors();
    void readGAM();
    void duringFlightOrientationTracking(double dt);
    void duringFlightPositionalTracking(double dt);
    void preFlightOrientationTracking(double dt);
    void preFlightPositionalTracking(double dt);
    void readBarometerData();
    void computeGravity();
};

// Global constants and objects declared as extern
extern const double GRAVITY;
extern craftState vehicle;
extern Adafruit_LIS3MDL magnetometer;
extern BMP388_DEV bmp388;

// Utility
double getDeltaTime();
#endif // SENSORFUSION_H
```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/servos.cpp

```cpp
/*
 * Program to control ARC-S Servo's
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 25, 2025
 * Edited:    November 17, 2025
 */

#include "servos.h"
#include "sensorfusion.h"

extern craftState vehicle;

int initialOffset = 1500;
String inputString = "";

AxisServo servos[] =
{
  {"+Y", Servo(), 3, initialOffset},
  {"-X", Servo(), 4, initialOffset},
  {"-Y", Servo(), 5, initialOffset},
  {"+X", Servo(), 6, initialOffset}
};

const int NUM_SERVOS = sizeof(servos) / sizeof(servos[0]);

void writeServoOffsetEEPROM(int servoIndex)
{
  int addr = servoIndex * 2;                  // 2 bytes per servo
  uint16_t val = servos[servoIndex].offset;  // current offset
  EEPROM.write(addr, val >> 8);              // high byte
  EEPROM.write(addr + 1, val & 0xFF);        // low byte
  EEPROM.commit();                             // save immediately
}

void loadOffsetsFromEEPROM()
{
  for (int i = 0; i < NUM_SERVOS; i++)
  {
    int addr = EEPROM_START + i * 2;
    uint16_t val = (EEPROM.read(addr) << 8) | EEPROM.read(addr + 1);
    if (val != 0xFFFF)
    { // ignore empty EEPROM
      servos[i].offset = val;
      servos[i].servo.writeMicroseconds(val); // center servo
      Serial.print("Loaded ");
      Serial.print(servos[i].name);
```

```
      Serial.print(" offset: ");
      Serial.println(val);
    }
  }
}

void processCommand(String cmd)
{
  cmd.trim();  // Remove whitespace
  if (cmd.length() == 0)
  {
    return;
  }

  int spaceIndex = cmd.indexOf(' ');
  if (spaceIndex == -1)
  {
    Serial.println("Invalid command. Use: ServoName Offset");
    return;
  }

  String servoName = cmd.substring(0, spaceIndex);
  String offsetStr = cmd.substring(spaceIndex + 1);
  int newOffset = offsetStr.toInt();

  bool found = false;
  for (int i = 0; i < NUM_SERVOS; i++)
  {
    if (servos[i].name == servoName)
    {
      servos[i].offset = newOffset;  // Update offset
      servos[i].servo.writeMicroseconds(newOffset);  // Move servo
      Serial.print("Updated ");
      Serial.print(servoName);
      Serial.print(" to offset ");
      Serial.println(newOffset);
      writeServoOffsetEEPROM(i);                    // Save to EEPROM
      found = true;
      break;
    }
  }
}

void readSerialAndUpdateServo()
{
  while (Serial.available() > 0)
  {
    char c = Serial.read();
    if (c == '\n')
```

```
    {    // End of input
      processCommand(inputString);
      inputString = "";               // Clear for next command
    }
    else
    {
      inputString += c;
    }
  }
}

void ControlServo(String name, float angle)
{
  for (int i = 0; i < NUM_SERVOS; i++)
  {
    if (servos[i].name == name)
    {
      int deltaUs = mapGyroToServo(angle, -90, 90);
      servos[i].writeDeltaUs(deltaUs);
      return;
    }
  }
}

float mapGyroToServo(float angle, float minAngle, float maxAngle)
{
  angle = constrain(angle, minAngle, maxAngle);

  return map(angle * 100,
             minAngle * 100, maxAngle * 100,
             -500, 500); // ±500 µs authority
}

void pitchYawControlLoop()
{
  // X-axis rotation controlled by servos on Y-axis
  ControlServo("+Y", vehicle.gyroAngle_deg.x);
  ControlServo("-Y", -1 * vehicle.gyroAngle_deg.x);

  // Y-axis rotation controlled by servos on X-axis
  ControlServo("+X", -1 * vehicle.gyroAngle_deg.y);
  ControlServo("-X", vehicle.gyroAngle_deg.x);
}

void rollControlLoop()
{
  // X-axis rotation controlled by servos on Y-axis
  //ControlServo("+Y", gyroPositionZAxis);
  //ControlServo("-Y", -1 * gyroPositionZAxis);
```

```
  ControlServo("+X", vehicle.gyroAngle_deg.z / 90); //90 DPS authority limitor
  ControlServo("-X", vehicle.gyroAngle_deg.z / 90);
}

```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/servos.h

```
/*
 * Header file for controling ARC-S Servo's
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 25, 2025
 * Edited:    November 17, 2025
 */

#ifndef SERVOS_H
#define SERVOS_H

#include <ESP32Servo.h>
#include <EEPROM.h>

#define EEPROM_SIZE 64   // adjust if you add more data
#define EEPROM_START 0   // start address

struct AxisServo
{
  String name;
  Servo servo;
  int pin;
  int offset; // center in microseconds

  void init_servo()
  {
    servo.setPeriodHertz(50);
    servo.attach(pin);
    servo.writeMicroseconds(offset); // center on init
  }

  void writeDeltaUs(int deltaUs)
  {
    int pulse = offset + deltaUs;
    pulse = constrain(pulse, 1500, 3000);
    servo.writeMicroseconds(pulse);
  }
};

void loadOffsetsFromEEPROM();
void readSerialAndUpdateServo();
void processCommand(String cmd);
void ControlServo(String name, float angle);
float mapGyroToServo(float angle, float minAngle, float maxAngle);
void pitchYawControlLoop();
void rollControlLoop();
extern const int NUM_SERVOS;
```

```
48 extern AxisServo servos[];
49 #endif
```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/telemetry.cpp

```cpp
/*
* cpp file ARC-S telemetry printouts
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 27, 2025
 * Edited:    February 27, 2025
 */

#include "Telemetry.h"
extern craftState vehicle;

void Telemetry::serialIMU()
{
    // CSV format: ACC_X,ACC_Y,ACC_Z
    Serial.print(vehicle.accel_g.x, 3); Serial.print(",");
    Serial.print(vehicle.accel_g.y, 3); Serial.print(",");
    Serial.println(vehicle.accel_g.z, 3);
}

void Telemetry::serialIMUDeadReckoning()
{
    // CSV format: DR_X,DR_Y,DR_Z
    Serial.print(vehicle.deadReckoning_mps2.x, 3); Serial.print(",");
    Serial.print(vehicle.deadReckoning_mps2.y, 3); Serial.print(",");
    Serial.println(vehicle.deadReckoning_mps2.z, 3);
}

void Telemetry::serialGyro()
{
    // CSV format: GYRO_X,GYRO_Y,GYRO_Z
    Serial.print(vehicle.gyro_dps.x, 2); Serial.print(",");
    Serial.print(vehicle.gyro_dps.y, 2); Serial.print(",");
    Serial.println(vehicle.gyro_dps.z, 2);
}

void Telemetry::serialGyroDeadReckoning()
{
    // CSV format: GP_X,GP_Y,GP_Z
    Serial.print(vehicle.gyroAngle_deg.x, 2); Serial.print(",");
    Serial.print(vehicle.gyroAngle_deg.y, 2); Serial.print(",");
    Serial.println(vehicle.gyroAngle_deg.z, 2);
}

void Telemetry::serialMagnometer()
{
    // CSV format: MAG_X,MAG_Y,MAG_Z
    Serial.print(vehicle.mag_uT.x, 2); Serial.print(",");
```

```
    Serial.print(vehicle.mag_uT.y, 2); Serial.print(",");
    Serial.println(vehicle.mag_uT.z, 2);
}

void Telemetry::serialBarometer()
{
    // CSV format: TEMP_C,PRESSURE_HPA,ALT_M
    Serial.print(vehicle.temperature_C, 2); Serial.print(",");
    Serial.print(vehicle.pressure_Pa, 2); Serial.print(",");
    Serial.println(vehicle.altitude_m, 2);
}

void Telemetry::serialServoOutputs()
{
    // CSV format: PITCH+Y,PITCH-Y,YAW+X,YAW-X,ROLL
    Serial.print(vehicle.gyroAngle_deg.x, 2); Serial.print(",");
    Serial.print(-vehicle.gyroAngle_deg.x, 2); Serial.print(",");
    Serial.print(-vehicle.gyroAngle_deg.y, 2); Serial.print(",");
    Serial.print(vehicle.gyroAngle_deg.y, 2); Serial.print(",");
    Serial.println(vehicle.gyroAngle_deg.z / 90, 2);
}

```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/telemetry.h

```
/*
 * Header file for controling ARC-S telemetry output settings
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 27, 2025
 * Edited:    January 27, 2025
 */

#pragma once
#include <Arduino.h>
#include <BMP388_DEV.h>
#include "sensorfusion.h"

class Telemetry {
public:
    void serialIMU();
    void serialIMUDeadReckoning();
    void serialGyro();
    void serialGyroDeadReckoning();
    void serialMagnometer();
    void serialBarometer();
    void serialServoOutputs();
};


```

~/Documents/GitHub/ARC-S-V5-Software/src/GroundStation/GroundStationGUI.py

```
1
2  """
3   * Python Code for ground station UI
4   *
5   * Name:      Thor Kopenkoskey
6   * Created:   January 25, 2025
7   * Edited:    November 17, 2025
8   """
9
10
11 import dearpygui.dearpygui as dpg
12 import serial
13 import collections
14 import time
15
16 ser = serial.Serial(
17     port='/dev/cu.usbserial-0001',
18     baudrate=115200,
19     timeout=1
20 )
21 Buffer = 300
22
23 data_buffers = {
24     "MX": collections.deque(maxlen=Buffer),
25     "MY": collections.deque(maxlen=Buffer),
26     "MZ": collections.deque(maxlen=Buffer),
27
28     "GX": collections.deque(maxlen=Buffer),
29     "GY": collections.deque(maxlen=Buffer),
30     "GZ": collections.deque(maxlen=Buffer)
31 }
32
33 TIME_WINDOW = 5
34
35 time_buffer = collections.deque(maxlen=Buffer)
36 start_time = time.time()
37
38 latest_value = [0.0]
39
40 def read_serial_data():
41     while ser.in_waiting > 0:
42         line = ser.readline().decode(errors='ignore').strip()
43
44
45         if ':' in line:
46             #Base setup
47             ID, val_str = line.split(':', 1)
```

```
            ID = ID.strip()
            val_str = val_str.strip()
            try:
                value = float(val_str)
            except ValueError:
                print(f"Ignored non-numeric line: {line}")
                return

            #Data type split
            if ID in data_buffers:
                data_buffers[ID].append(value)

                current_time = time.time() - start_time  # Time since start
                time_buffer.append(current_time)

dpg.create_context()

with dpg.window(label="Mango Serial Plot", width=450, height=450, pos=(0,0)):
    dpg.add_text("Reading serial data...")
    dpg.add_separator()

    with dpg.plot(label="Axis Data (MX, MY, MZ)", height=400, width=-1):
        dpg.add_plot_legend()
        dpg.add_plot_axis(dpg.mvXAxis, label="Time", tag="x_axis_M")
        dpg.add_plot_axis(dpg.mvYAxis, label="Value", tag="y_axis_M")

        line_series = dpg.add_line_series([], [], label="MX", parent="y_axis_M",
tag="MX_data_series")
        line_series = dpg.add_line_series([], [], label="MZ", parent="y_axis_M",
tag="MZ_data_series")
        line_series = dpg.add_line_series([], [], label="MY", parent="y_axis_M",
tag="MY_data_series")

        dpg.set_axis_limits("y_axis_M", -200, 200)


with dpg.window(label="Gyro Serial Plot", width=450, height=450, pos=(450,0)):
    dpg.add_text("Reading serial data...")
    dpg.add_separator()

    with dpg.plot(label="Axis Data (GX, GY, GZ)", height=400, width=-1):
        dpg.add_plot_legend()
        dpg.add_plot_axis(dpg.mvXAxis, label="Time", tag="x_axis_G")
        dpg.add_plot_axis(dpg.mvYAxis, label="Value", tag="y_axis_G")

        line_series = dpg.add_line_series([], [], label="GX", parent="y_axis_G",
tag="GX_data_series")
        line_series = dpg.add_line_series([], [], label="GZ", parent="y_axis_G",
tag="GZ_data_series")
```

```
        line_series = dpg.add_line_series([], [], label="GY", parent="y_axis_G",
tag="GY_data_series")

        dpg.set_axis_limits("y_axis_G", -200, 200)


def update_display():
    read_serial_data()

    if len(time_buffer) > 0:
        current_time = time_buffer[-1]
        min_time = max(0, current_time - TIME_WINDOW)

        # Filter to only show recent data within the time window
        recent_indices = [i for i, t in enumerate(time_buffer) if t >= min_time]
        recent_times = [time_buffer[i] for i in recent_indices]

        for axis in ["MX", "MY", "MZ", "GX", "GY", "GZ"]:
            # REMOVE the synchronization check - it's too strict
            # Just check if we have any data at all
            if len(data_buffers[axis]) > 0:
                # Make sure we don't try to access out-of-range indices
                valid_indices = [i for i in recent_indices if i <
len(data_buffers[axis])]
                valid_times = [time_buffer[i] for i in valid_indices]
                recent_values = [data_buffers[axis][i] for i in valid_indices]

                if len(valid_times) > 0 and len(recent_values) > 0:
                    dpg.set_value(f"{axis}_data_series", [valid_times,
recent_values])

        # Update X-axis limits to create scrolling effect
        dpg.set_axis_limits("x_axis_M", min_time, current_time)
        dpg.set_axis_limits("x_axis_G", min_time, current_time)




dpg.create_viewport(title='Experimental ground station', width=1200, height=1200)
dpg.setup_dearpygui()
dpg.show_viewport()


while dpg.is_dearpygui_running():
    update_display()
    dpg.render_dearpygui_frame()
    time.sleep(0.001)

dpg.destroy_context()
```

~/Documents/GitHub/ARC-S-V5-Software/src/GroundStation/websocket.cpp

```
/*
 * Websocket File for communication with tskopen.com, Future web ground station UI for portability and multi computer access.
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 25, 2025
 * Edited:    November 17, 2025
 */


/*
Compile command:
clang++ -std=c++17 -o websocket websocket.cpp \
  -I/opt/homebrew/include \
  -I/opt/homebrew/opt/boost/include \
  -L/opt/homebrew/opt/boost/lib \
  -lpthread
  */

#include <boost/beast/core.hpp>
#include <boost/beast/websocket.hpp>
#include <boost/asio/connect.hpp>
#include <boost/asio/ip/tcp.hpp>


#include <boost/asio.hpp>
#include <boost/asio/ssl.hpp>
#include <boost/beast.hpp>
#include <boost/beast/ssl.hpp>

#include <cstdlib>

#include <iostream>
#include <string>

namespace http = beast::http;           // from <boost/beast/http.hpp>
namespace websocket = beast::websocket; // from <boost/beast/websocket.hpp>
namespace ssl   = net::ssl;
using namespace std;

namespace net = boost::asio;
namespace beast = boost::beast;
using namespace boost::beast;
using namespace boost::beast::websocket;

net::io_context ioc;
tcp_stream sock(ioc);
```

```
net::ssl::context ctx(net::ssl::context::tlsv12);
using tcp = boost::asio::ip::tcp;       // from <boost/asio/ip/tcp.hpp>


class WebSocketClient
{
public:
  string host;
  string port;
  string text;
WebSocketClient(string host, string port, string text) : host_(host), port_(port), ws(ioc_) {}

  string handshakeServer()
  {
    try
    {

stream<net::ssl::stream<tcp_stream>> wss(net::make_strand(ioc), ctx);
        tcp::resolver resolver{ioc};
        websocket::stream<tcp::socket> ws{ioc};

           boost::asio::ip::tcp::resolver resolver(ioc_);
           auto const results = resolver.resolve(host_, port_);



net::connect(ws.next_layer().next_layer(), results);

wss.next_layer().handshake(net::ssl::stream_base::client);
         std::cout << "Connected!" << std::endl;

ws.write(boost::asio::buffer(text_));

        beast::flat_buffer buffer;
        ws.read(buffer);
        string received_message = beast::buffers_to_string(buffer.data());
        ws.close(websocket::close_code::normal);

        return received_message;
    }

    catch(std::exception const& e)
    {
        return e.what();
    }

  }
private:
```

```
  string host_;
  string port_;
  string text_;
  boost::asio::io_context ioc_;
  ssl::context ctx_{ssl::context::sslv23_client};
  tcp::socket socket_{ioc_};
};

int main()
{
        WebSocketClient client("websocket.tskopen.com", "443", "test");

        string response = client.handshakeServer();

        std::cout << response << std::endl;

        return 0;
}
```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/flash.cpp

```
/*
* Program to control ARC-S Flash Chip
 */

#include <Arduino.h>
#include <SPI.h>

#define FLASH_CS    36
#define FLASH_CLK   35
#define FLASH_MISO  37
#define FLASH_MOSI  34

SPIClass spiFlash(HSPI);

void setupFlash()
{
    pinMode(FLASH_CS, OUTPUT);
    digitalWrite(FLASH_CS, HIGH);

    spiFlash.begin(FLASH_CLK, FLASH_MISO, FLASH_MOSI, FLASH_CS);

    Serial.println("External flash SPI bus ready");
}
```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/flash.h

```
/*
 * Header file for ARC-S Flash Chip Controller
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 15, 2025
 * Edited:    November 15, 2025
 */

#include <Preferences.h>


void setupFlash();

```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/sdcard.cpp

```cpp
/*
 * Program to control ARC-S SD Card logging
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 11, 2025
 * Edited:    January 15, 2025
 */
#include "sdcard.h"
#include <Arduino.h>
#include <SPI.h>
#include <SD.h>

#define SDCardCS    18
#define SDCardCLK   35
#define SDCardMISO  12
#define SDCardMOSI  16

void setupSDCard()
{
    // Initialize SPI bus for SD card
    SPI.begin(SDCardCLK, SDCardMISO, SDCardMOSI, SDCardCS);

    // Initialize SD card
    if (!SD.begin(SDCardCS))
    {
        Serial.println("ERROR: SD card initialization failed");
    }
    else
    {
        Serial.println("SD card ready");
    }
}


```

~/Documents/GitHub/ARC-S-V5-Software/src/flightCompV2/sdcard.h

```
/*
 * Header file for ARC-S SD Card logging Controller
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 11, 2025
 * Edited:    January 15, 2025
 */

#include <Preferences.h>


void setupSDCard();

void dataLogSDCard(const char *logFile, const char *data);

```